

8.1. Chapitre 2 — Les variables : la mémoire de la console : ce qui vient

L'entrée du chapitre 2 : ce qu'on a vu jusqu'ici, et ce que ses 4 étapes vont ajouter, une à la fois.

1. Ce qu'il faut comprendre

D'où l'on vient. Le chapitre 1, « Les tout premiers pas », vient de se terminer. Tout ce qu'il a montré reste valable : on s'en sert à partir d'ici sans le réexpliquer.

Ce que ce chapitre apporte : Les variables : la mémoire de la console. Il compte 4 étapes, dont 4 venues du cours :

- Une variable, une case qui porte un nom
- Calculer : + - * / et %
- Un octet s'arrête à 255
- Où vit une variable : avant la boucle, ou dedans

Les fonctions qui arrivent : `#include <multiplier>`. Chacune sera présentée seule, juste avant la première étape qui l'emploie : jamais deux lignes nouvelles d'un coup.

Comment avancer : une étape à la fois. Lis l'explication, lance le programme, regarde ce qu'on doit voir, puis fais l'essai proposé. Si quelque chose t'échappe, l'étape d'avant contient la pièce qui manque.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **432 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         29 o ← démarrage - ta fonction
VBlank         8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        160 o ← CopierTuiles - 10 lettres sur 43
"CHAPITRE 2"   10 o ← texte() - un texte gravé
"C EST PARTI"  11 o ← texte() - un texte gravé
Lettres gravées : ACEHIPRST2
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
```

Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() × 2 22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1 3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

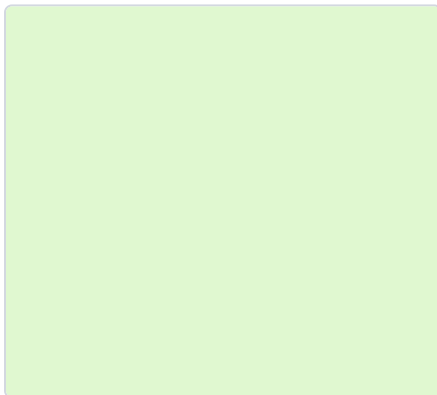
```
// ---- Chapitre 2 : Les variables : la mémoire de la console ----  
// L'entrée du chapitre : rien de neuf dans ce programme. Il annonce la suite.  
  
#include <texte>    // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

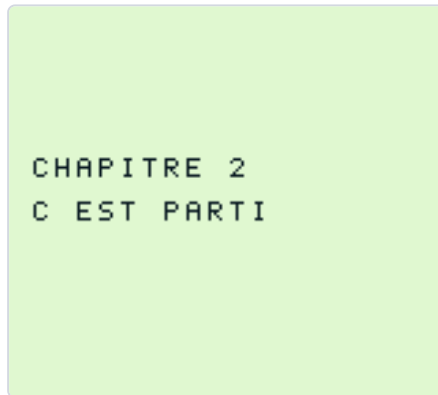
```
int main() {  
    texte(1, 7, "CHAPITRE 2");    // le numéro du chapitre  
    texte(1, 9, "C EST PARTI");  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

CHAPITRE 2, puis C EST PARTI.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le chapitre 2 s'annonce

